

FocusAid

Catching what a Deaf student misses when they look away — and bridging it, in the moment.

A Deaf or hard-of-hearing student in a lecture is doing two jobs with one channel. They are **reading** — captions, an interpreter, a notetaker — and they are **watching**: the slide, the diagram, the teacher's hands. Vision cannot do both at once. Every glance down to read costs the board. Every glance up to watch costs the words.

THE PROBLEM WE PICKED

A hearing student never has to solve this. They listen *while* looking, because audio is a parallel channel. For our user that channel is closed, so attention becomes a queue instead of a stream — and in a live lecture there is no rewind.

So the gap is not captions. Most lecture halls already have them. The gap is **the thirty seconds after attention breaks**: the student knows they lost something, does not know what, and the class has already moved on. That specific, repeated, invisible loss is what FocusAid exists to close.

WHAT IT DOES

1. **Transcribes the lesson live** from the room's audio, or reads a video lesson's captions.
2. **Watches attention** through the webcam — gaze direction, brow-furrow confusion, and head pose — entirely in the browser.
3. **Detects a missed window** when attention drops for a sustained stretch, and marks it on the lesson timeline.
4. **Bridges it** — one tap surfaces what was actually said while they were away, as a short summary and a diagram.

There is also a manual “*I’m looking away*” button, because the student always knows before the camera does.

WHY THIS, AND NOT ANOTHER CAPTIONING TOOL

Accessibility tooling tends to stop at *delivery*: get the words on screen and the job is done. That assumes the student was available to receive them. The interesting, unsolved part is **recovery** — and recovery is only possible if something is tracking attention and content on the same clock. That is the product we wanted to build, and it is why attention tracking and transcription are one system here rather than two features.

WHAT MAKES IT DIFFERENT

1. It closes a loop. Most tools display. FocusAid notices a gap, identifies what fell into it, and offers it back. Detection and content share a single timeline, so “what was said at 2:14” is unambiguous.

2. We measured our AI instead of demoing it.

We built a small language model into the browser (SmolLM2-360M, WebGPU) to rewrite missed passages in plainer language. Before trusting it, we tested it against our lesson transcript. It was faithful on **0 of 6 passages**:

"the Calvin cycle gets oxygen from the air"	it takes carbon dioxide
"RuBisCO helps fix some of that sugar "	it fixes carbon
"chlorophyll in your cells "	wrong organism
"we'll see the light-dependent reactions "	the passage said light- <i>independent</i>

A model nearly twice the size (Qwen2.5-0.5B) was no better — only differently wrong, inventing a toy-car analogy for mitosis.

This mattered more to us than it would to most teams, for one reason: **a hearing student catches these errors against the audio. Ours cannot.** The entire reason they are reading our text is that the audio channel is closed to them. An accessibility tool that hallucinates is not merely inaccurate — it exploits the exact dependency it created.

So we changed the architecture rather than the prompt. **The on-device model is no longer allowed to write.** It only *selects* which of the teacher's own sentences matters most, and every selection is verified against the source transcript before it reaches the screen. Anything that is not verbatim is discarded. Measured: 5 of 6 windows now produce a verified key point, and the sixth shows nothing rather than guessing. The model can still be wrong about *importance*, which the student recovers from in a second. It can no longer be wrong about *facts*.

3. It degrades instead of failing. Summaries are built from the transcript itself and render instantly; AI is layered on top as an upgrade, never placed between the student's tap and an answer. No network, no API key, no WebGPU, no microphone — each removes capability without removing the product. Attention tracking and the on-device model run entirely in the browser, so a student's webcam feed and lesson audio never leave their machine.

WHERE IT STANDS

Working end to end in the browser: live transcription, real webcam attention tracking, missed-window detection, catch-up summaries with generated diagrams, a searchable transcript with persistent notes, and full keyboard and screen-reader support. The AI layer runs in three tiers — hosted model when a key is configured, on-device model when WebGPU is available, and a deterministic summary built from the teacher's words always.

FocusAid — built in 5 hours by four developers working in parallel. Next.js 16, React 19, MediaPipe, transformers.js. See the technical brief for implementation detail.